Pokročilé objektové technológie

6

ASP.NET Core

API a gRPC

KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Úloha 1 – pripojte sa na REST API

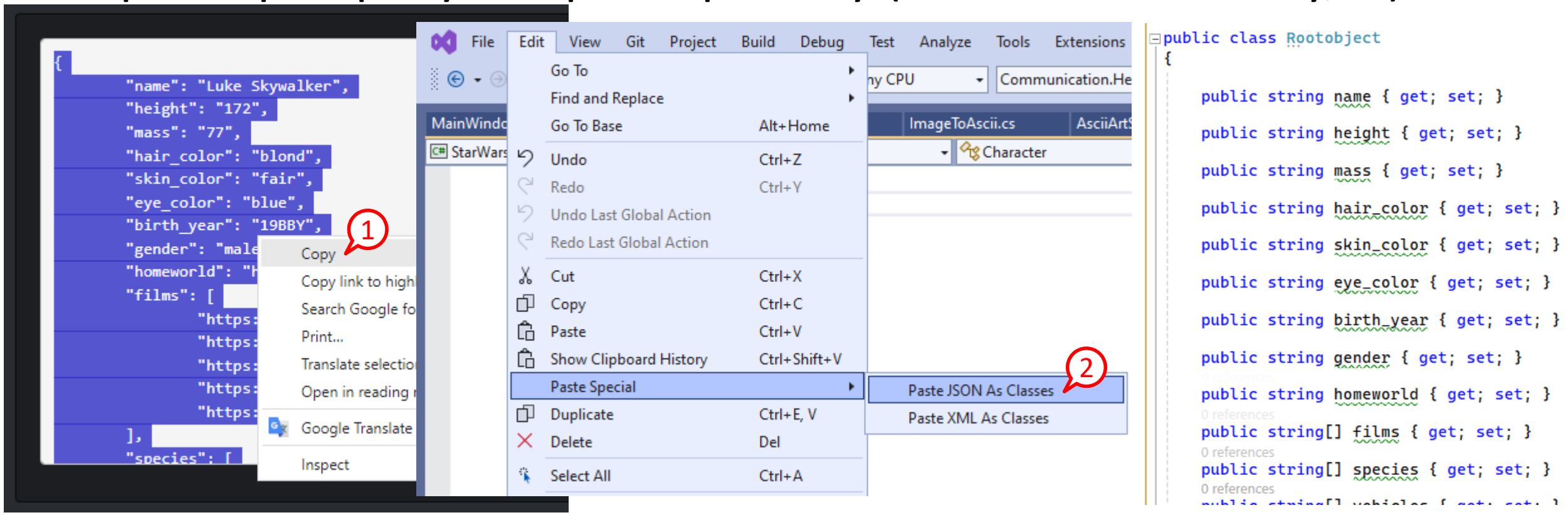
- Vytvorte konzolovú aplikáciu (projekt typu Console App), v ktorej sa pripojíte na existujúce REST API pomocou triedy HttpClient:
  - SWAPI (Star Wars API) – https://swapi.dev/
    - https://swapi.dev/api/people/{id}
    - https://swapi.dev/api/people/?search={text}

```
static async Task<Character?> GetCharacterById(int id)
{
    try
    {
        using var httpClient = new HttpClient();
        var response = await httpClient.GetAsync($"https://swapi.dev/api/people/{id}");
        return response.IsSuccessStatusCode ? await response.Content.ReadFromJsonAsync<Character>() : null;
    }
    catch (Exception ex)
    {
        Console.Error.WriteLine("Error: " + ex.Message);
        return null;
    }
}
```

```
public class Character
{
    public string Name { get; set; }
    public string Height { get; set; }
    public string Mass { get; set; }
}
```

Možnosť vygenerovať C# triedy z JSON objektov (1)

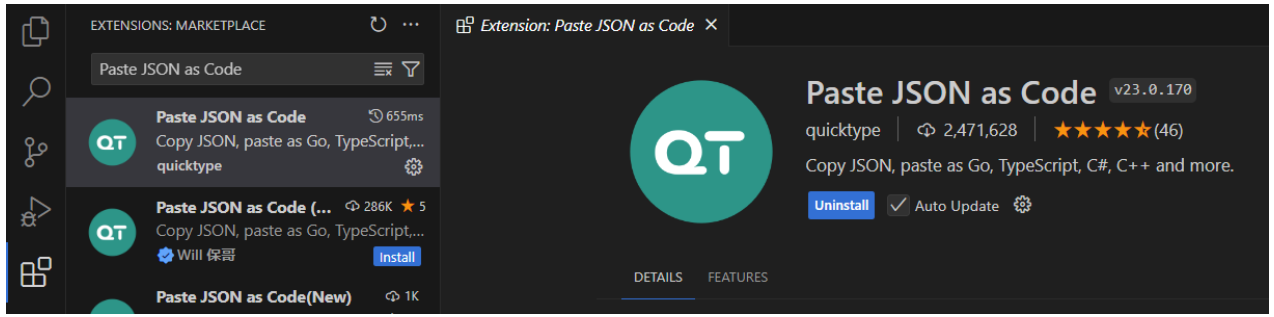
- **Visual Studio 2022** – skopírovať JSON objekt do schránky:
  - Edit -> Paste Special -> Paste JSON As Classes
- Upraviť prilepený kód podľa potreby (zmeniť názov triedy, …)

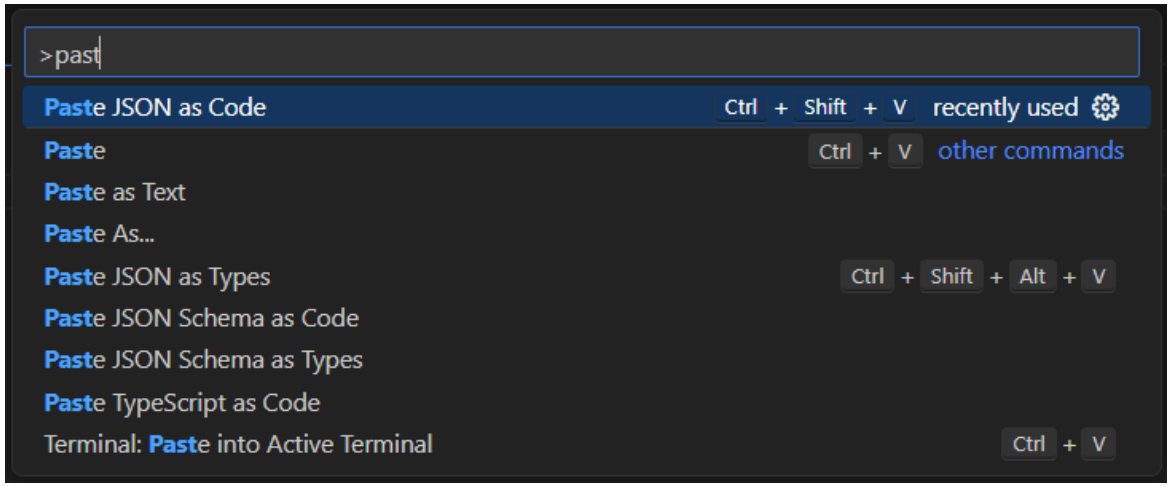


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Možnosť vygenerovať C# triedy z JSON objektov (2)

- **Visual Studio Code** – nutné mať rozšírenie:
  - Napr. **Paste JSON as Code** (quicktype.quicktype)






```
// <auto-generated />
//
// To parse this JSON data, add NuGet 'Newtonsoft.Json' then do:
//
//    using QuickType;
//
//    var topLevel = TopLevel.FromJson(jsonString);

namespace QuickType
{
    using System;
    using System.Collections.Generic;

    using System.Globalization;
    using Newtonsoft.Json;
    using Newtonsoft.Json.Converters;

    4 references
    public partial class TopLevel
    {
        [JsonProperty("name")]
        0 references
        public string Name { get; set; }

        [JsonProperty("height")]
        [JsonConverter(typeof(ParseStringConverter))]
        0 references
        public long Height { get; set; }

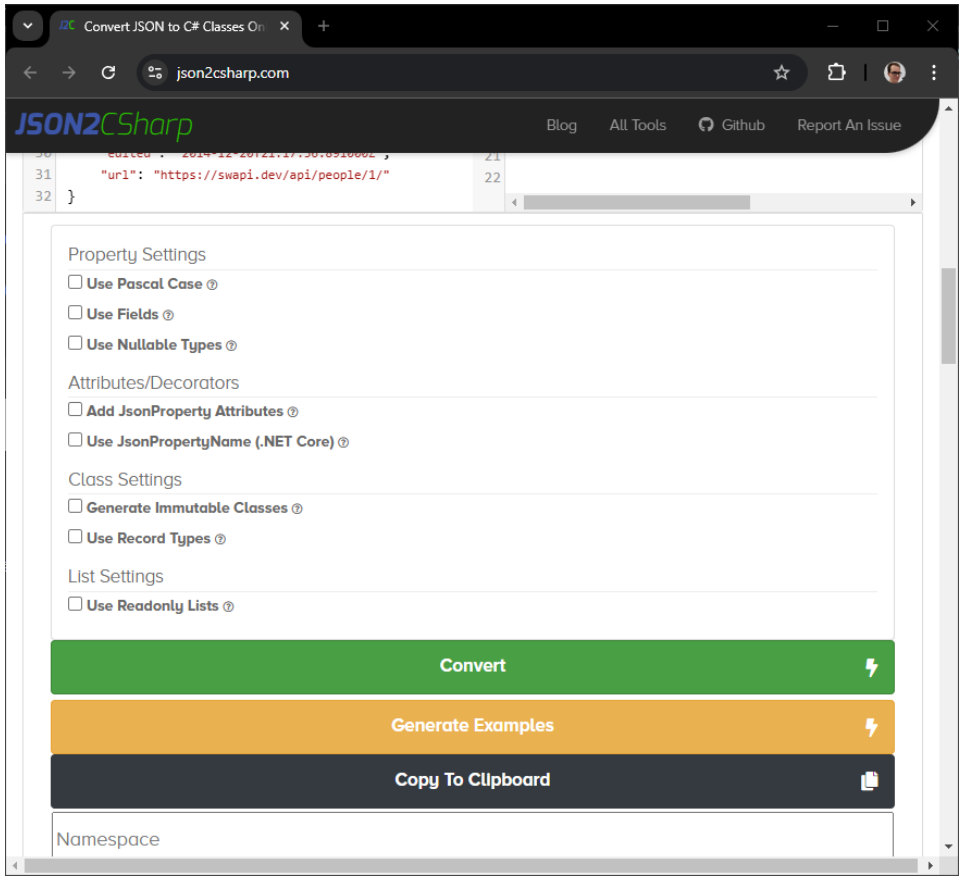
        [JsonProperty("mass")]
        [JsonConverter(typeof(ParseStringConverter))]
        0 references
        public long Mass { get; set; }

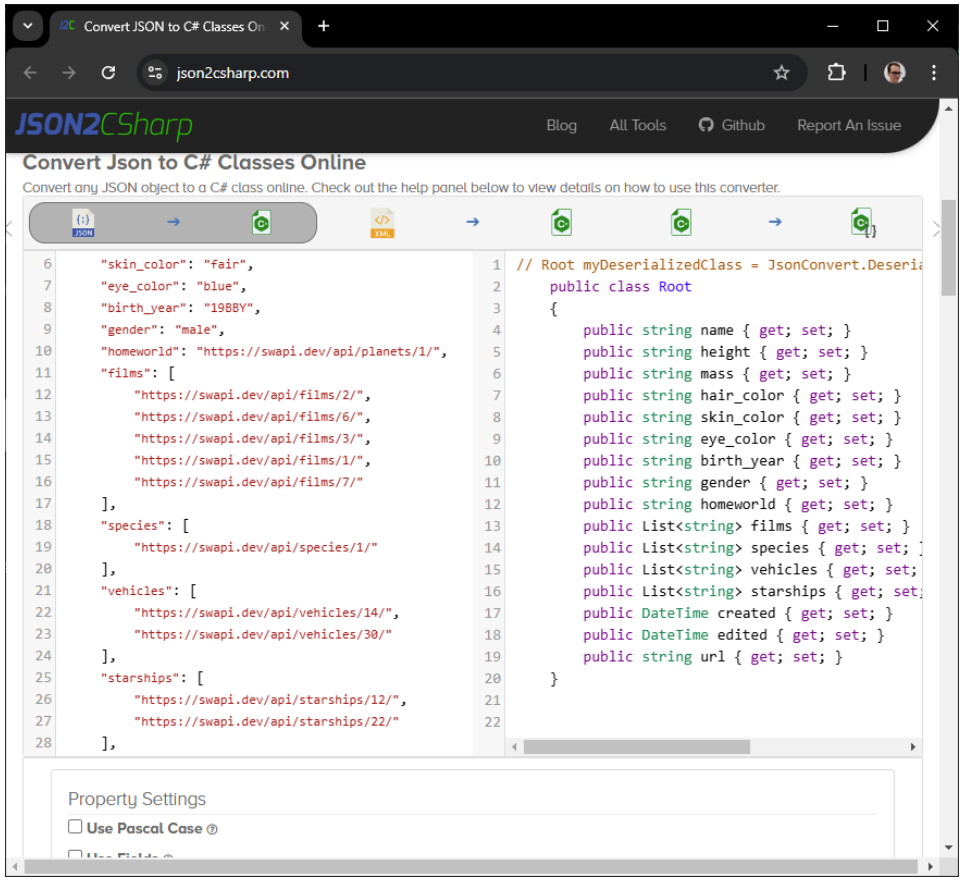
        [JsonProperty("hair_color")]
        0 references
        public string HairColor { get; set; }

        [JsonProperty("skin_color")]
```

Možnosť vygenerovať C# triedy z JSON objektov (3)

- Webové nástroje:
  - Napr. https://json2csharp.com/





Vyhľadávanie textu

```
private async Task<SearchCharacterResult?> GetCharactersBySearch(string text)
{
    try
    {
        using var httpClient = new HttpClient();
        var response = await httpClient.GetAsync($"https://swapi.dev/api/people/?search={text}");
        if (response.IsSuccessStatusCode)
        {
            return await response.Content.ReadFromJsonAsync<SearchCharacterResult>(
                //new System.Text.Json.JsonSerializerOptions()
                //{
                //    PropertyNamingPolicy = JsonNamingPolicy.SnakeCaseLower // Iba v .NET 8 namiesto [JsonPropertyName("hair_color")]
                //}
            );
        }
        else
        {
            return null;
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"An error occurred: {ex.Message}", "Error",
            MessageBoxButton.OK, MessageBoxImage.Error);
        return null;
    }
}
```

```
public class SearchCharacterResult
{
    public int Count { get; set; }
    public Character[] Results { get; set; }
}

public class Character
{
    public string Name { get; set; }
    public string Height { get; set; }
    public string Mass { get; set; }

    [JsonPropertyName("hair_color")]
    public string HairColor { get; set; }

    //...
```

Dokumentácia

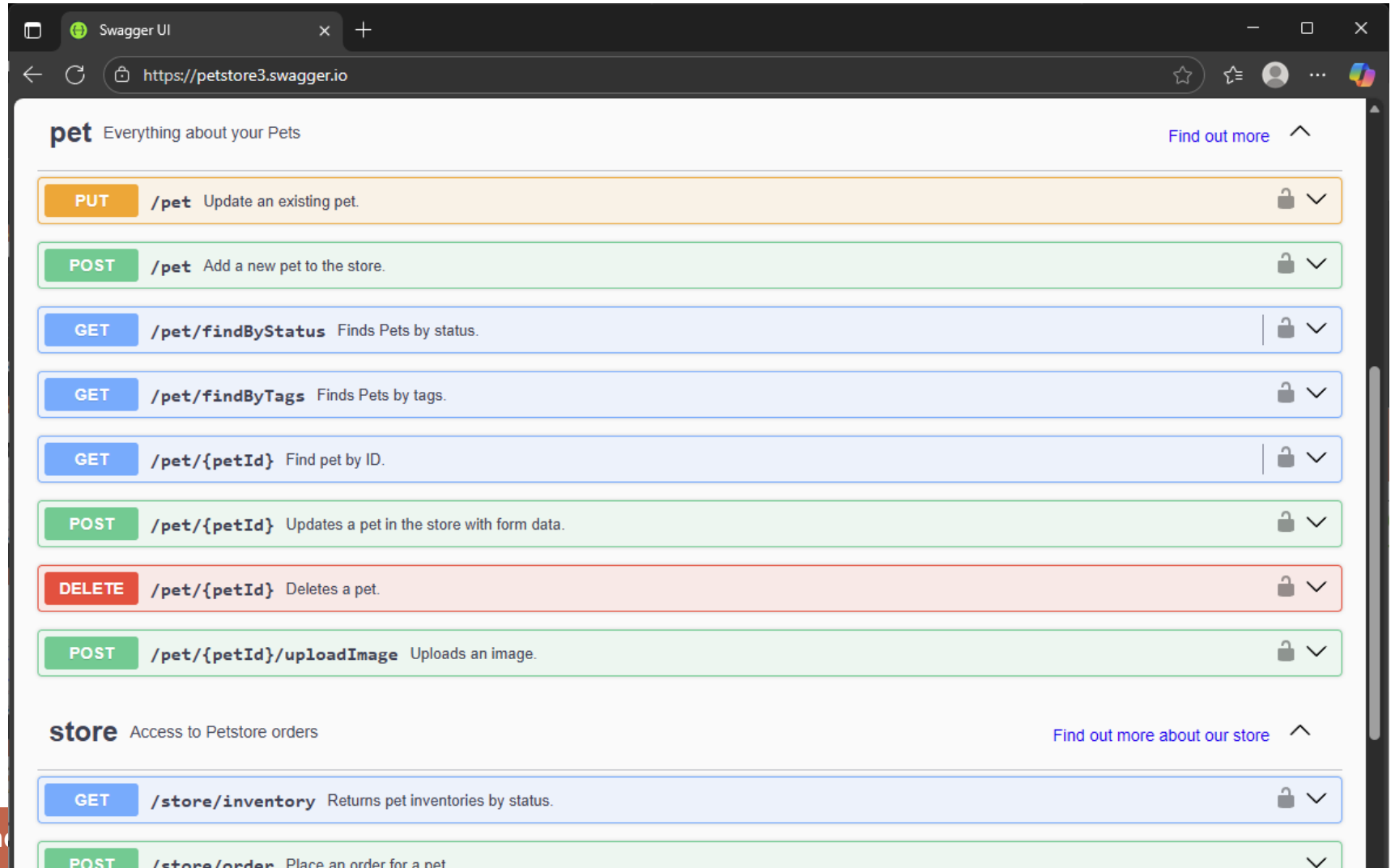
- https://swapi.dev/documentation má síce popísanú dokumentáciu a mnohé pomocné knižnice, avšak chýba jej OpenAPI špecifikácia
  - Napr. pre C#: https://github.com/M-Yankov/SWapi-CSharp – dostupný cez NuGet balíček https://www.nuget.org/packages/SWapiCSharp/

- Iný podobný projekt https://swapi.online/ poskytuje dokumentáciu cez Swagger – OpenAPI otvorená špecifikácia pre REST API:
  - https://swapi.online/api-docs/ – **vyskúšajte si**
  - https://swapi.online/api/swagger.json



Úloha 1.1

- Preskúmajte API na adrese https://petstore3.swagger.io/

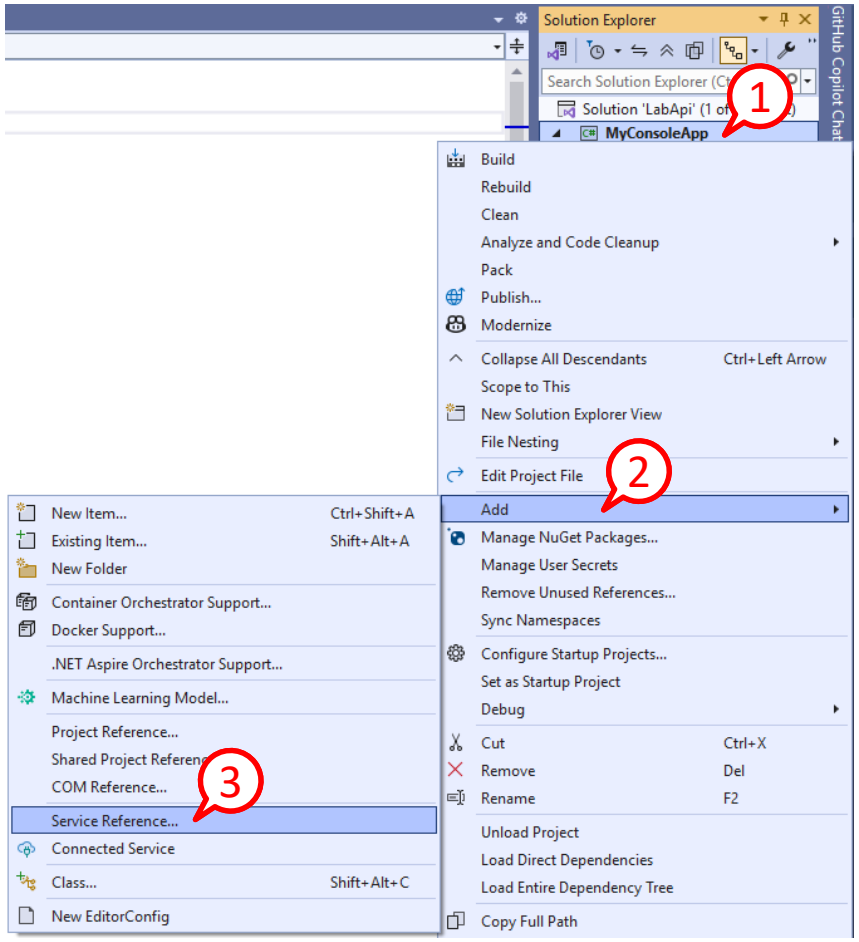


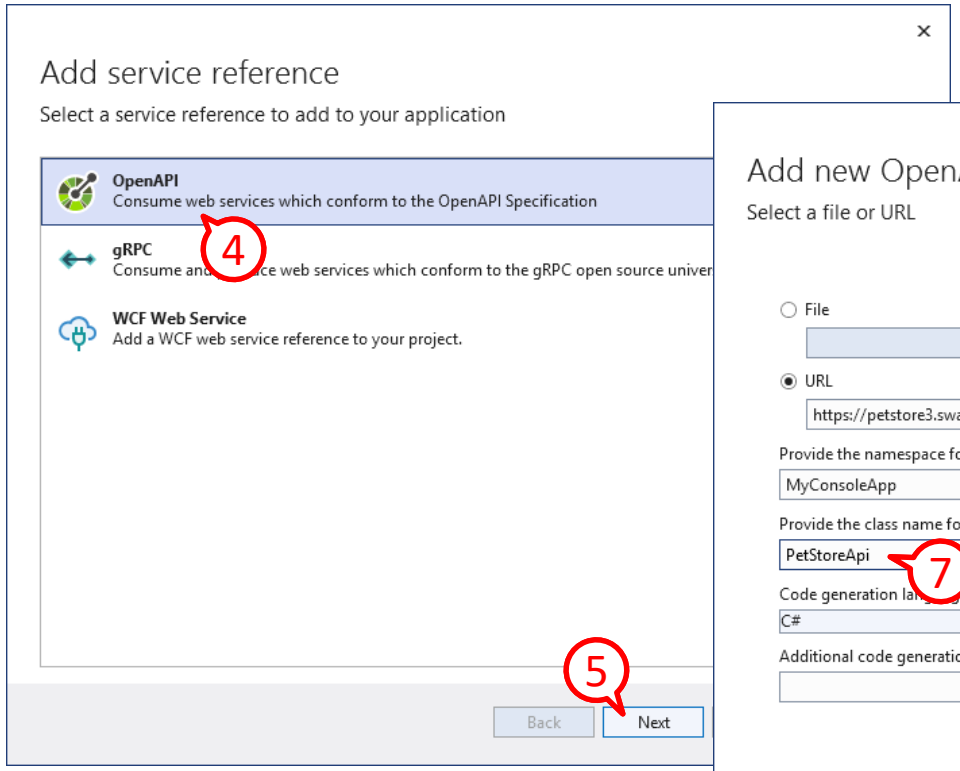
Úloha 1.2 – Visual Studio 2022 – referencia na službu

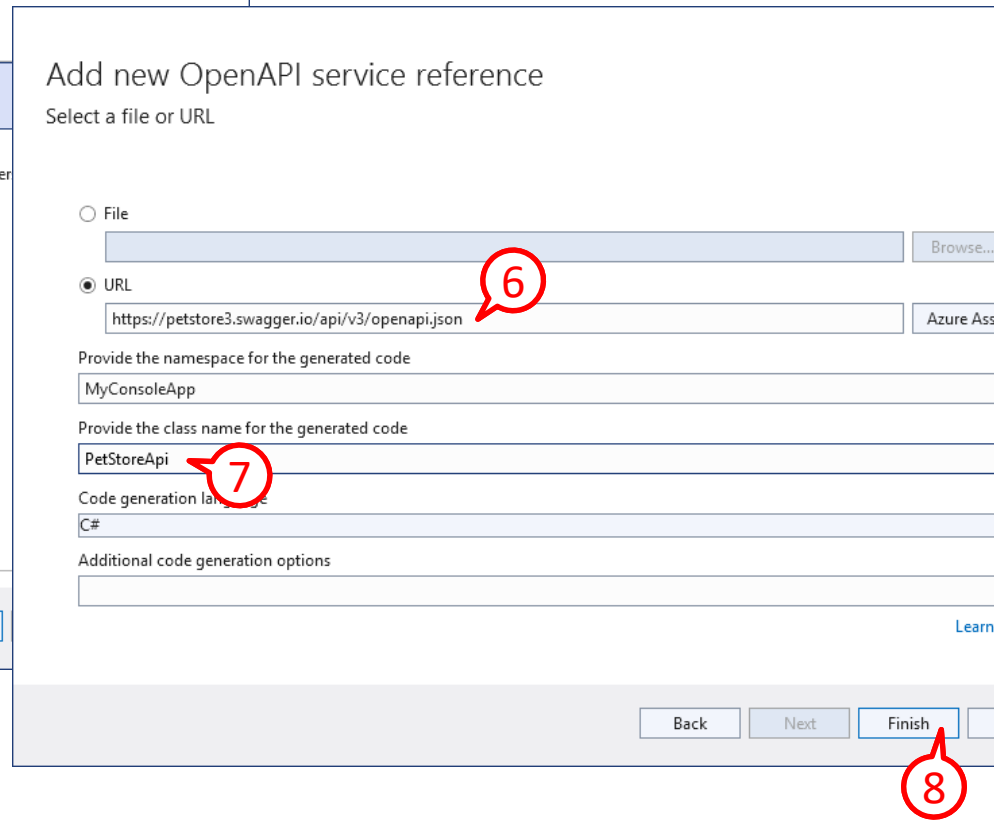
- Pridajte si referenciu cez Visual Studio 2022 na službu https://petstore3.swagger.io/api/v3/openapi.json cez **Add -> Service Reference…** a nechajte si vygenerovať automatickú triedu na prácu s API

Visual Studio 2022 – referencia na službu (1)

- Ak máte k dispozícii OpenAPI formát v súbore typu **.json** alebo **.yaml**, môžete si nechať vygenerovať

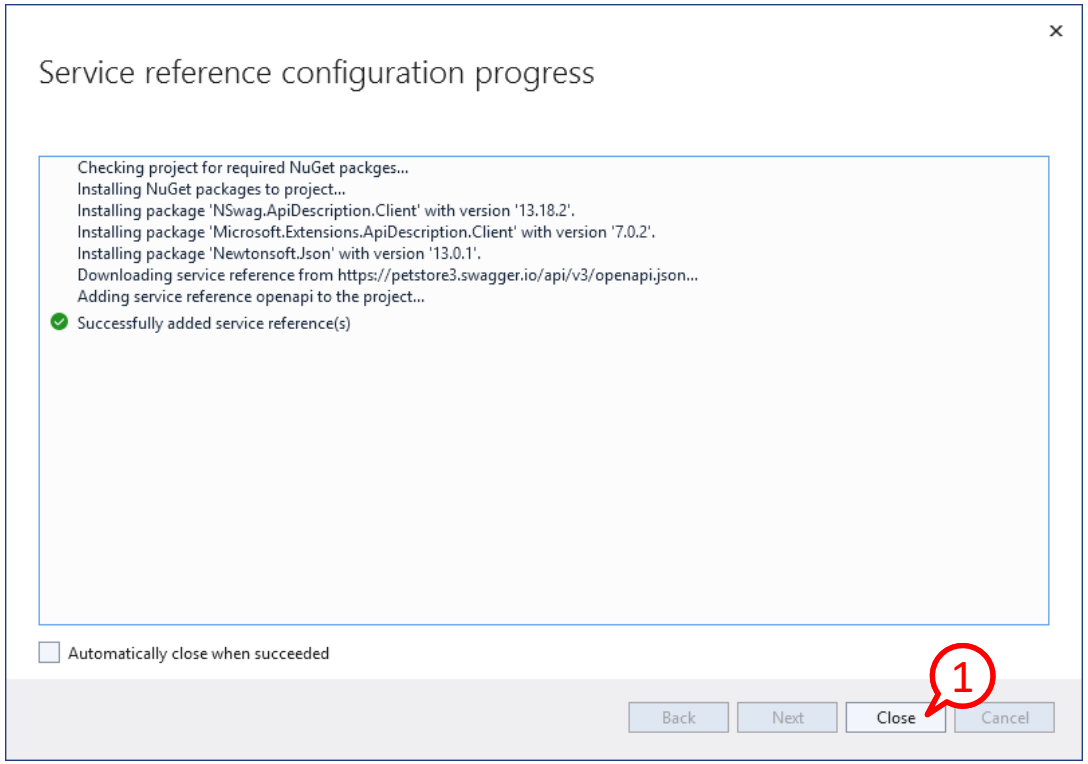


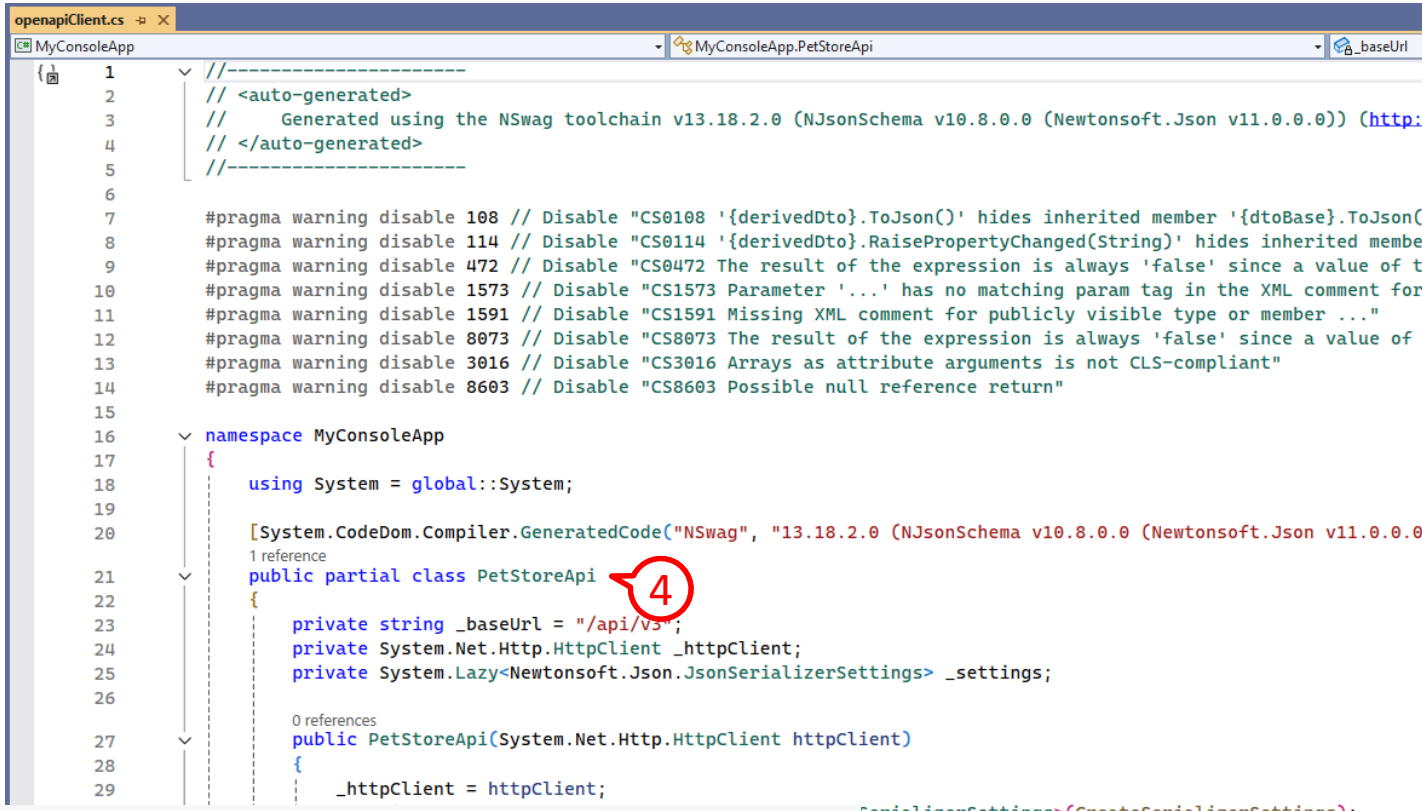


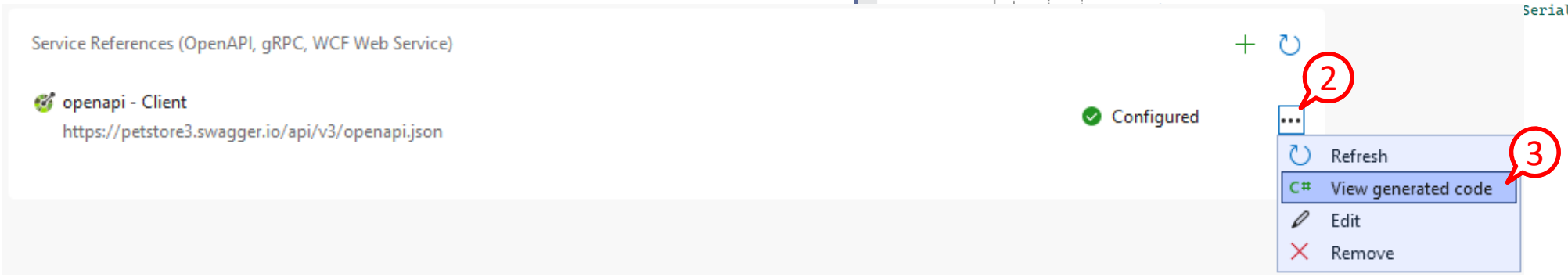


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Visual Studio 2022 – referencia na službu (2)







Visual Studio 2022 – referencia na službu (3)

- Konzumácia služby:

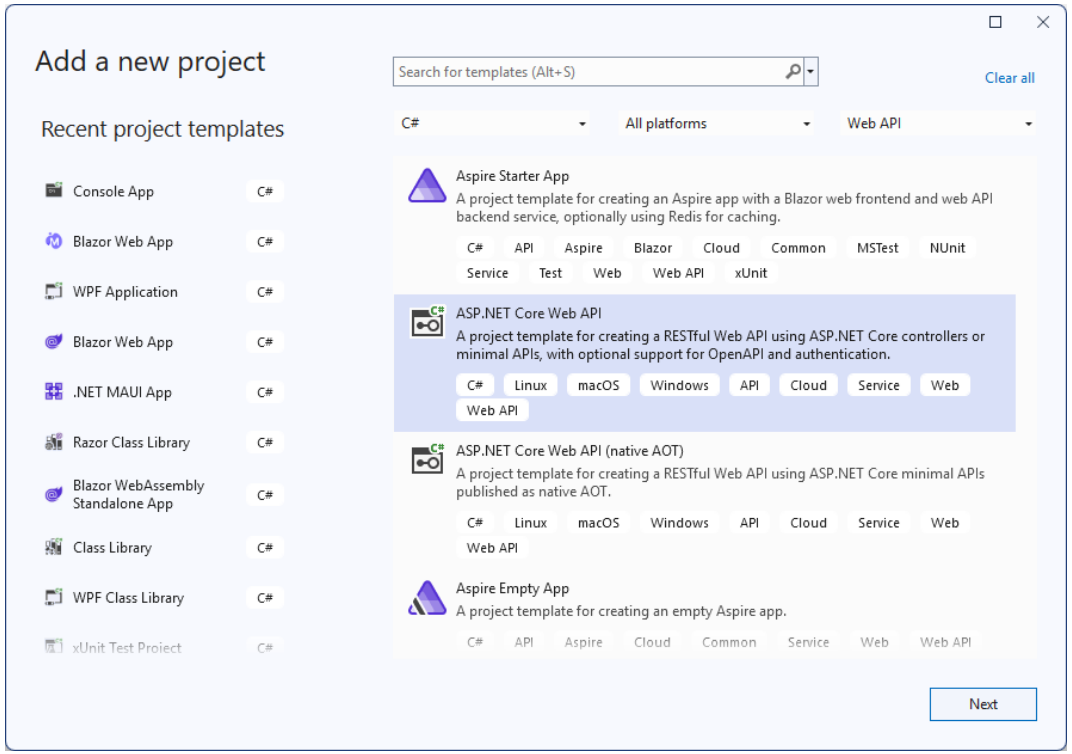
```
var http = new HttpClient
{
    BaseAddress = new Uri("https://petstore3.swagger.io")
};

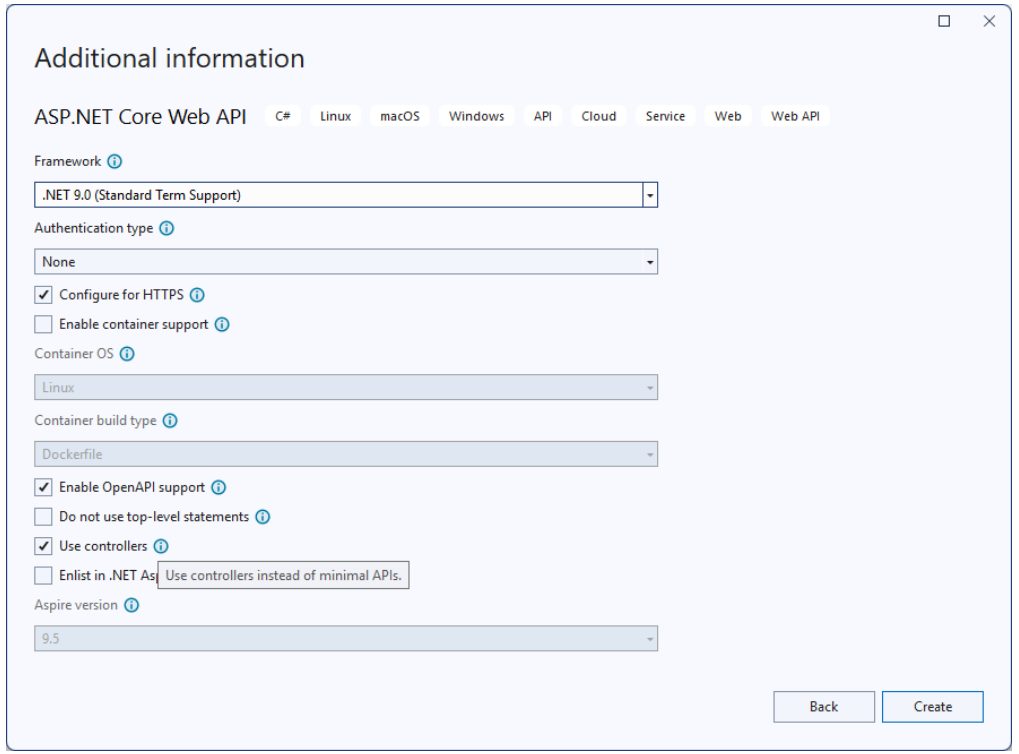
var api = new PetStoreApi(http);
var pet = await api.GetPetByIdAsync(1);
Console.WriteLine(pet.Name);
```

- Poznámka: referencia na službu nie vždy správne funguje – musíte mať kvalitný a dobre popísaný .json aj so schémami, inak to nie je veľmi použiteľné – napr. v prípade https://swapi.online/api/swagger.json nie sú definované návratové typy

Úloha 2 – vytvorte si vlastné Web API

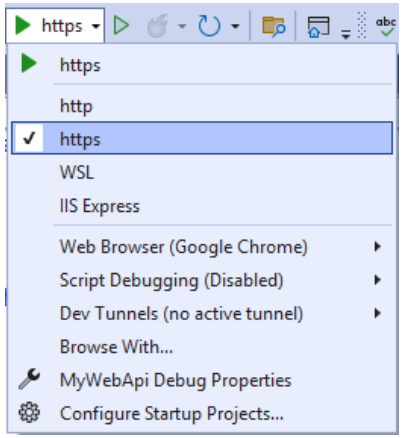
- Vytvorte nový projekt typu **ASP.NET Core Web API**

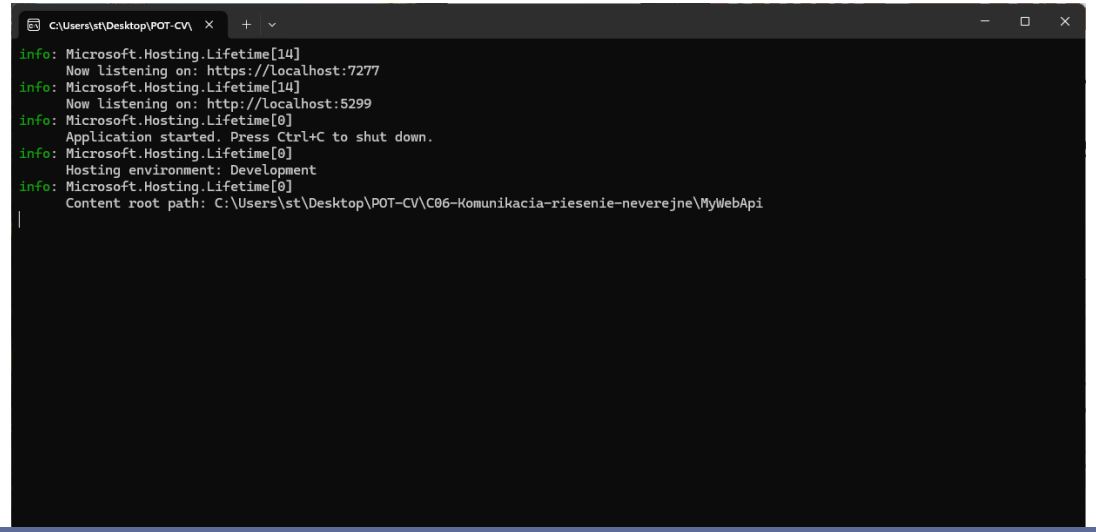


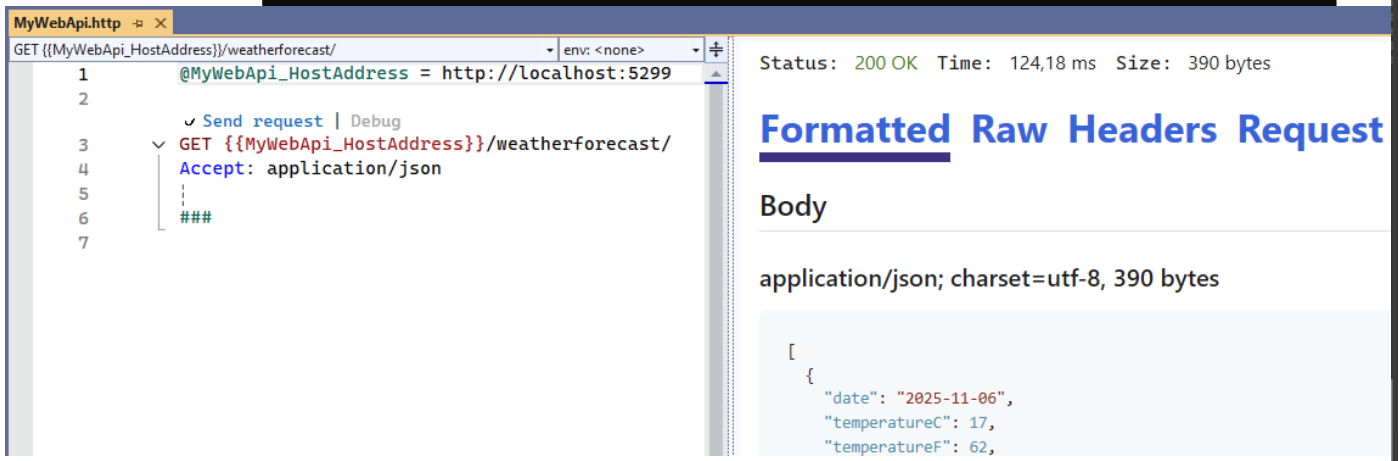


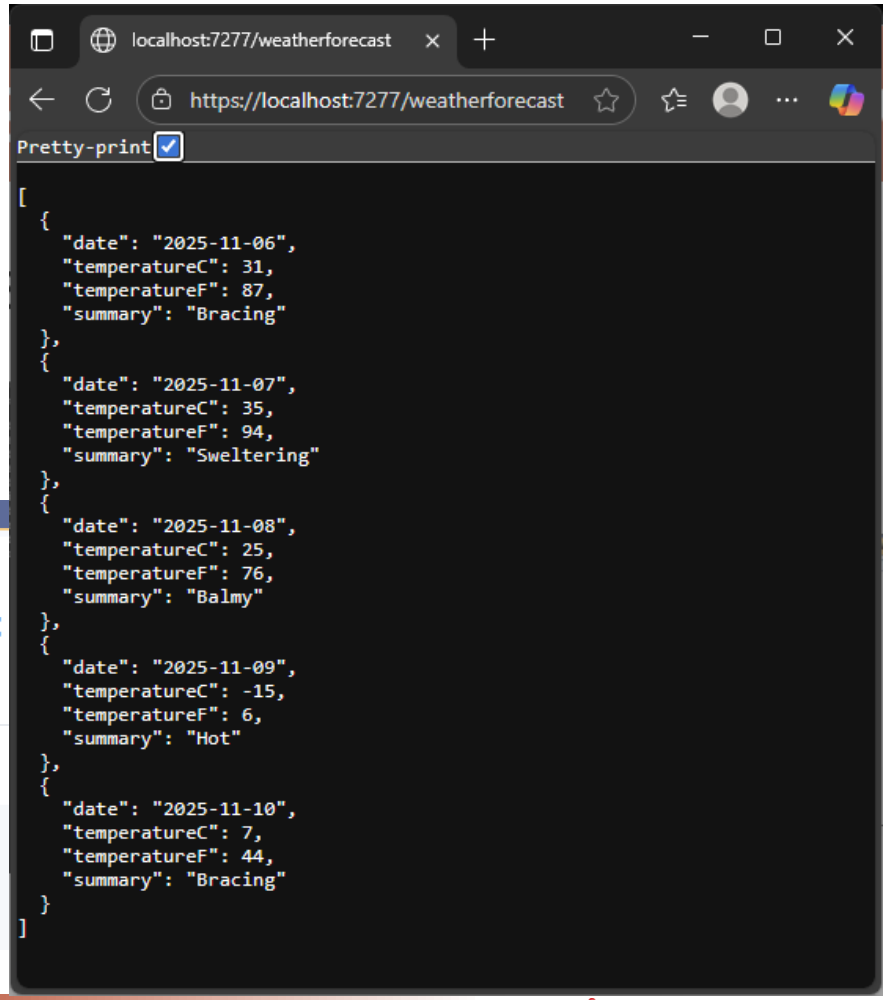
Úloha 2.1 – spustite a otestujte web API (1)

- Skompilujte, spustite projekt a otestujte endpoint **weatherforecast**



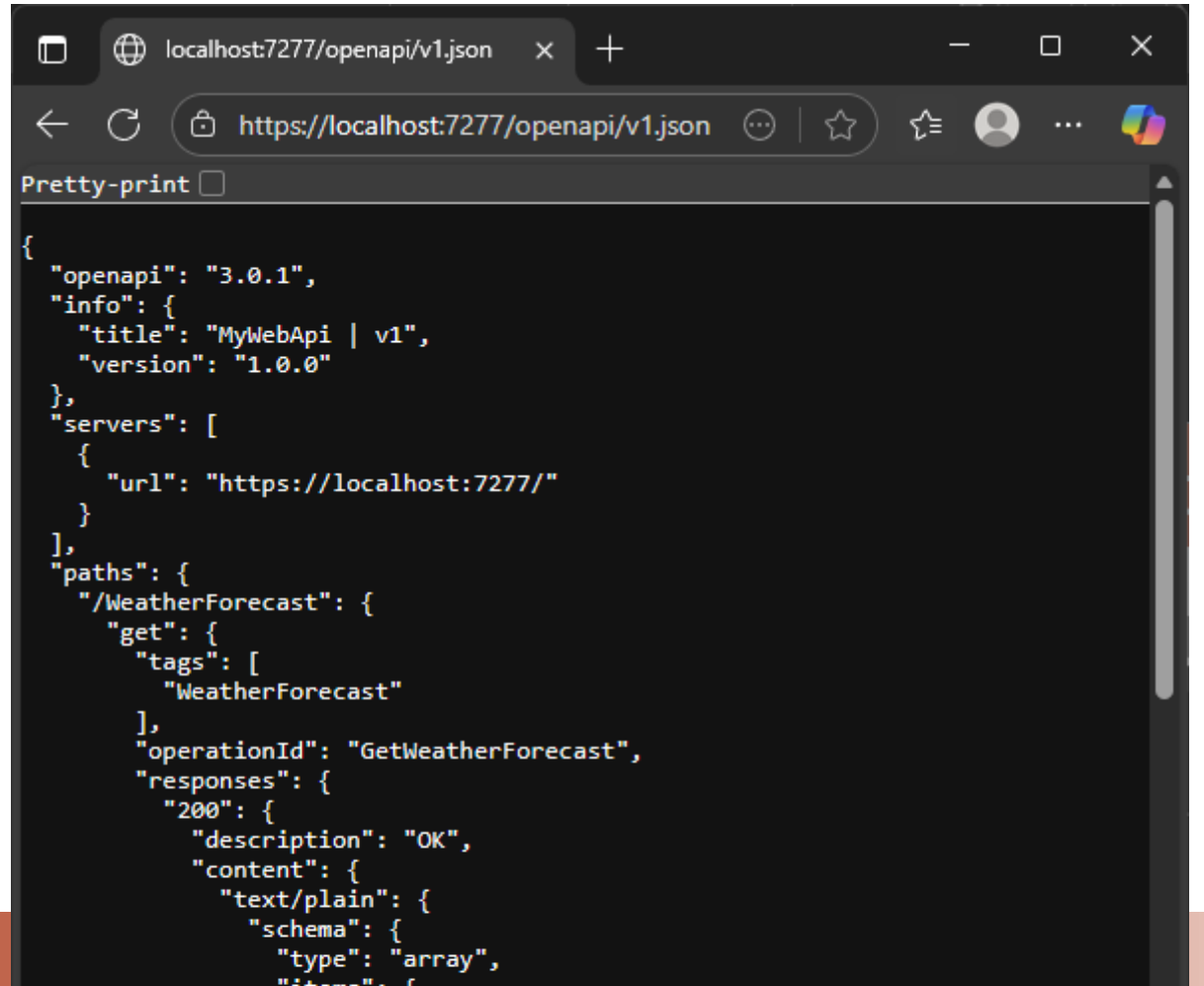






Úloha 2.1 – spustite a otestujte web API (2)

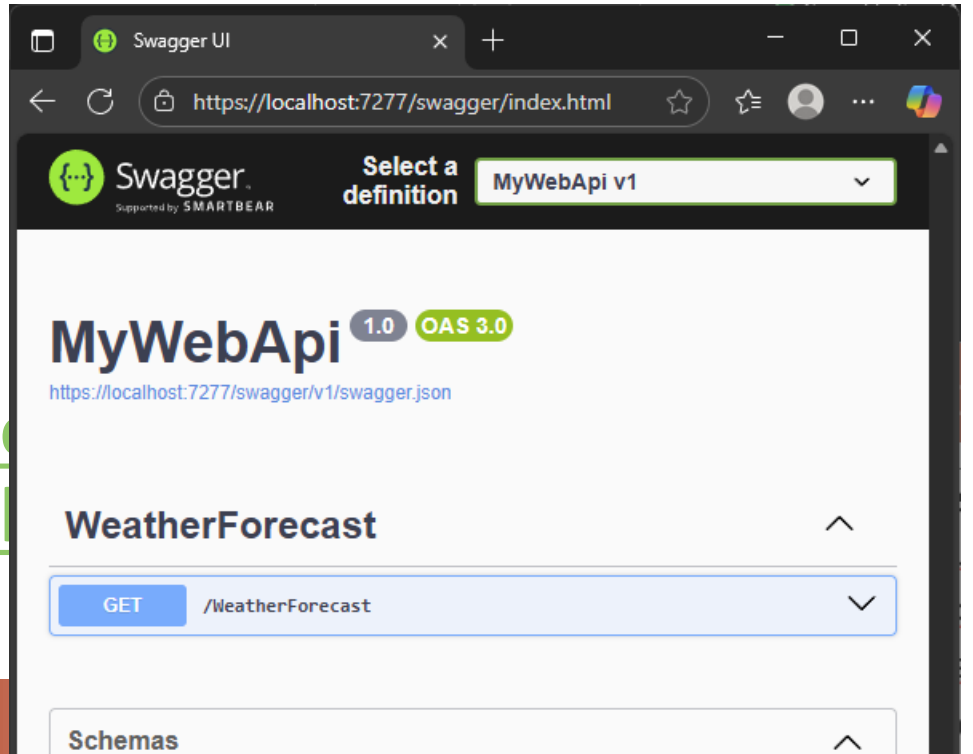
- V projekte je definovaný **app.MapOpenApi();**, vďaka ktorému sa môžeme dostať k OpenAPI formátu na adrese **openapi/v1.json**:



Úloha 2.2 – pridajte Swagger

- Pridajte si NuGet balíček **Swashbuckle.AspNetCore** a následne si pridajte pred builder.Build():
  - builder.Services.AddEndpointsApiExplorer();
  - builder.Services.AddSwaggerGen();

  a po builder.Build() do app.Environment.IsDevelopment() :
  - app.UseSwagger();
  - app.UseSwaggerUI();
- Následne môžete spustiť projekt a prejsť na adresu: https://localhost:7277/swagger
- Pre viac informácií: https://learn.microsoft.c
  us/aspnet/core/tutorials/getting-started-wit



Úloha 3 – vytvorte si API s minimal API

- Vytvorte druhý projekt typu **ASP.NET Core Web API**
  - odškrtnite políčko **Use controllers**, aby ste využili jednoduchšie „minimal APIs“ (ak nepotrebujete veľa endpointov, minimal API stačí)
  - **Porovnajte kód** v Program.cs v tomto projekte so súbormi Program.cs a WeatherForecastController.cs z predchádzajúceho projektu

Úloha 3.1 – pridajte si triedy do projektu

- Pridajte si predpripravené priložené zdrojové kódy:

```
record Book(Guid Id, string Title, string Author, int Year);
class InMemoryBooks
{
    //...
}
class CreateBookDto
{
    [Required, MinLength(2)] public string? Title { get; set; }
    [Required, MinLength(2)] public string? Author { get; set; }
    [Range(1450, 2100)] public int? Year { get; set; }
}
class UpdateBookDto
{
    [MinLength(2)] public string? Title { get; set; }
    [MinLength(2)] public string? Author { get; set; }
    [Range(1450, 2100)] public int? Year { get; set; }
}
```

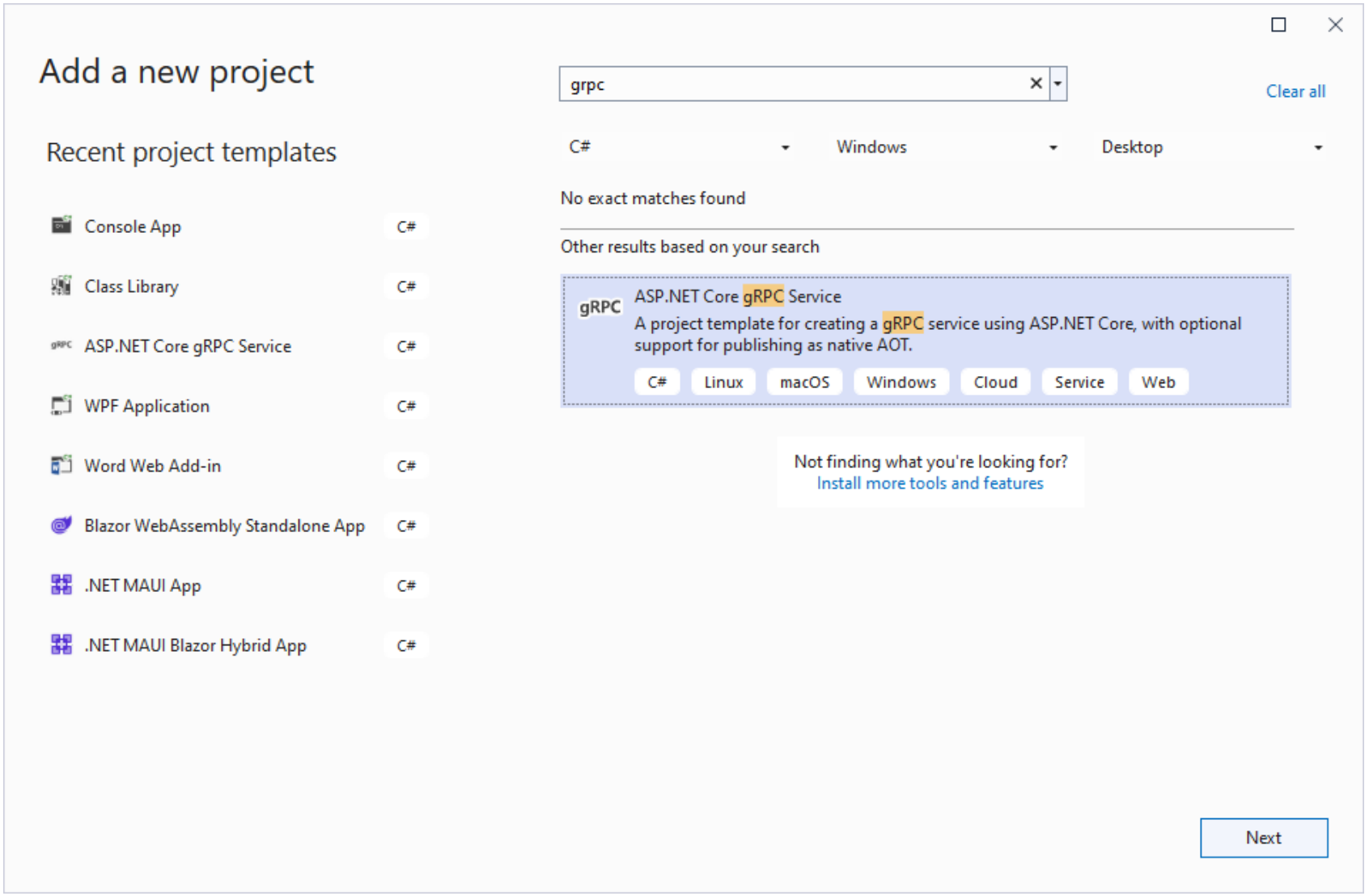
Úloha 3.2 – vytvorte API pre správu kníh

- Vytvorte REST rozhranie, ktoré bude pracovať s inštanciou InMemoryBooks db = new(); a implementuje nasledujúce endpointy:
  - **GET /api/books** – vráti všetky knihy
  - **GET /api/books/{id}** – vráti konkrétnu knihu podľa Guid id. Ak neexistuje, vráti HTTP 404 NotFound
  - **POST /api/books** – vytvorí novú knihu podľa CreateBookDto, validácia: [Required], [Range], [MinLength]. Ak validácia neprejde, vráti HTTP 400 ValidationProblem. Ak prejde, vráti HTTP 201 Created
  - **PUT /api/books/{id}** – upraví existujúcu knihu podľa UpdateBookDto. Ak kniha neexistuje, vráti HTTP 404 NotFound. Ak validácia neprejde, vráti HTTP 400 ValidationProblem. Ak sa úspešne upraví, vráti HTTP 204 NoContent
  - **DELETE /api/books/{id}** – vymaže knihu podľa id. Ak neexistuje, vráti 404 NotFound. Ak sa úspešne vymaže, vráti 204 NoContent

Úloha 4 – vytvorte vlastnú gRPC službu

- Vytvorte nový projekt typu **ASP.NET Core gRPC Service** a skompilujte ho
- Preskúmajte vo vytvorenom projekte nasledujúce súbory:
  - **greet.proto** – obsahuje definíciu gRPC **Greeter** služby s metódou **SayHello** a správami **HelloRequest**, **HelloReply**, ktoré sa serializujú cez Protocol Buffers (Protobuf) – na základe tohto súboru sa vygenerujú triedy pre službu a správy
  - ***.csproj** – projektový súbor obsahujúci:
    - NuGet balíček **Grpc.AspNetCore** (zahŕňa balíčky Google.Protobuf a Grpc.Tools)
    - **greet.proto** súbor s nastavením atribútu **GrpcServices** pre generovanie triedy služby (hodnota Server): `<Protobuf Include="Protos\greet.proto" GrpcServices="Server" />`
  - **GreeterService.cs** – pozrite sa (napr. pomocou F12) na vygenerovanú triedu predka **Greeter.GreeterBase** v obj priečinku vďaka nastaveniu v *.csproj súbore
  - **launchSettings.json** – pozrite si port v *applicationUrl* – každý nový vygenerovaný projekt má rôzne vygenerovaný port, môžete ho zmeniť

Šablóna gRPC služby



KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Úloha 5 – vytvorte klienta pre gRPC službu

- Vytvorte novú aplikáciu, ktorá sa pripojí na gRPC službu na Vašej URL adrese
- Aby ste mohli gRPC službu „konzumovať“, potrebujete:
  - NuGet balíčky:
    - **Grpc.Net.Client** – obsahuje triedy pre prácu s gRPC službou
    - **Google.Protobuf** – obsahuje triedy pre serializáciu správ cez Protocol Buffers
    - **Grpc.Tools** – obsahuje kompilátor, ktorý generuje triedy z *.proto súborov
  - ***.proto** súbor – konkrétne **greet.proto**
  - do súboru ***.csproj** pridať zahrnutie greet.proto súboru s nastavením atribútu **GrpcServices** pre generovanie triedy **klienta** (hodnota Client):

```
<Protobuf Include="Protos\greet.proto" GrpcServices="Client" />
```

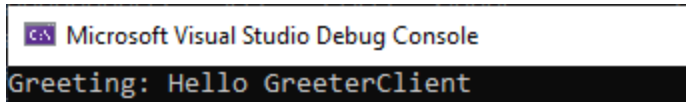
- Poznámka:
  - Vďaka Grpc.Tools sa na základe proto súboru vygenerujú v obj priečinku súbory s triedami, ktoré môžeme použiť na pripojenie – konkrétne **Greeter.GreeterClient**

Klient gRPC služby Greeter

- V konzolovej aplikácii:

```
using var channel = GrpcChannel.ForAddress("https://localhost:7047");
var client = new Greeter.GreeterClient(channel);
var reply = await client.SayHelloAsync(new HelloRequest { Name = "GreeterClient" });
Console.WriteLine("Greeting: " + reply.Message);
```

- Výstup:



Úloha 6 – AsciiArt

- Vytvorte novú službu **AsciiArtService**, ktorá bude generovať z obrázkov ASCII art texty (využite priloženú triedu ImageToAscii)

```
service AsciiArt {
  rpc ConvertImageToAscii (ImageRequest) returns (AsciiArtResponse);
}

message ImageRequest {
    bytes imageData = 1;
}

message AsciiArtResponse {
    string asciiArt = 1;
}
```



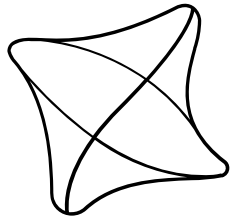


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Upozornenie

- Tieto študijné materiály sú určené výhradne pre študentov predmetu **Pokročilé objektové technológie** na Fakulte riadenia a informatiky Žilinskej univerzity v Žiline

- Reprodukovanie, šírenie (i častí) materiálov bez písomného súhlasu autora nie je dovolené



ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Ing. **Štefan Toth**, PhD.
Katedra softvérových technológií
Fakulta riadenia a informatiky
Žilinská univerzita v Žiline
stefan.toth@fri.uniza.sk